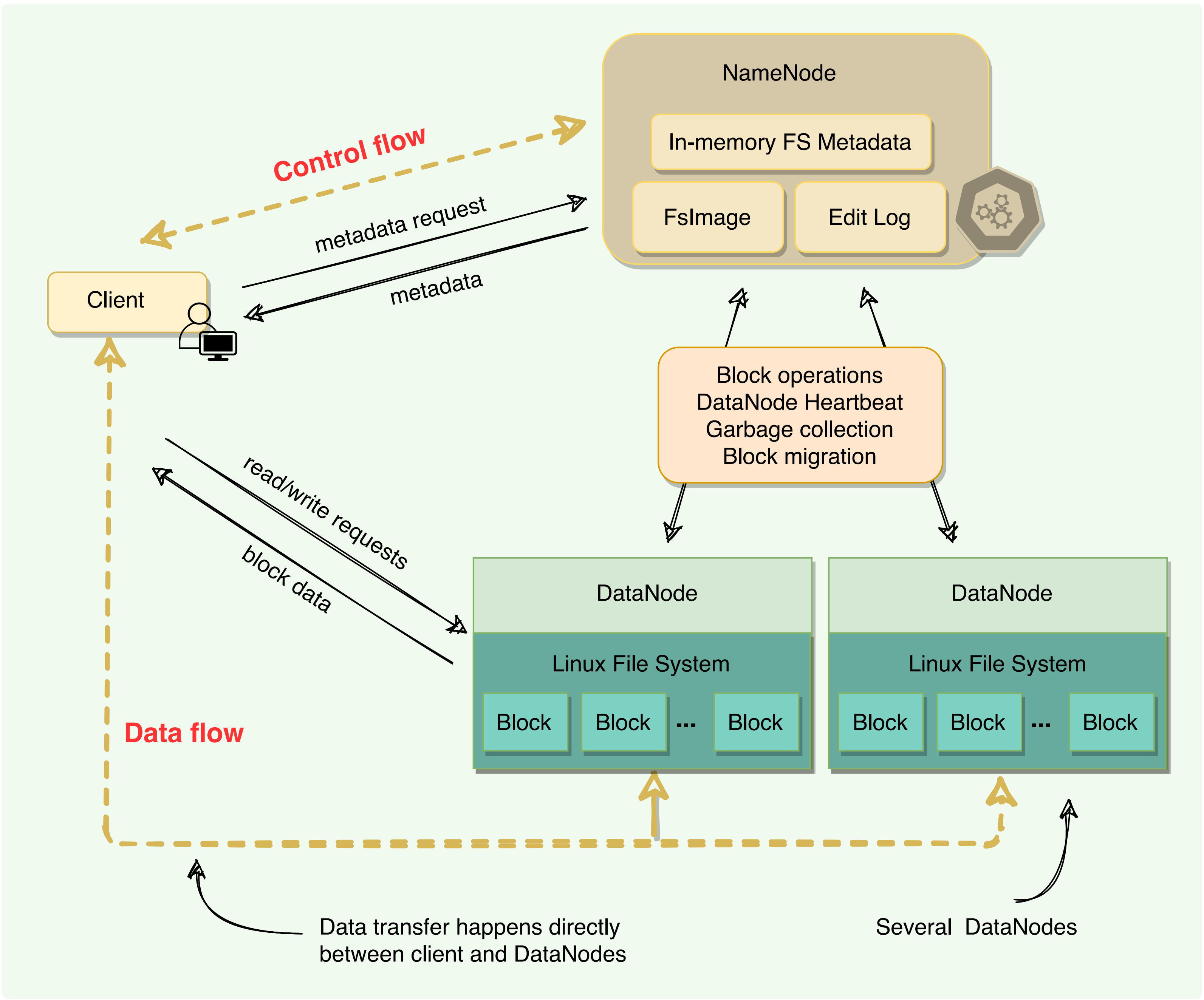
High-level Architecture

This lesson gives a brief overview of HDFS's architecture.

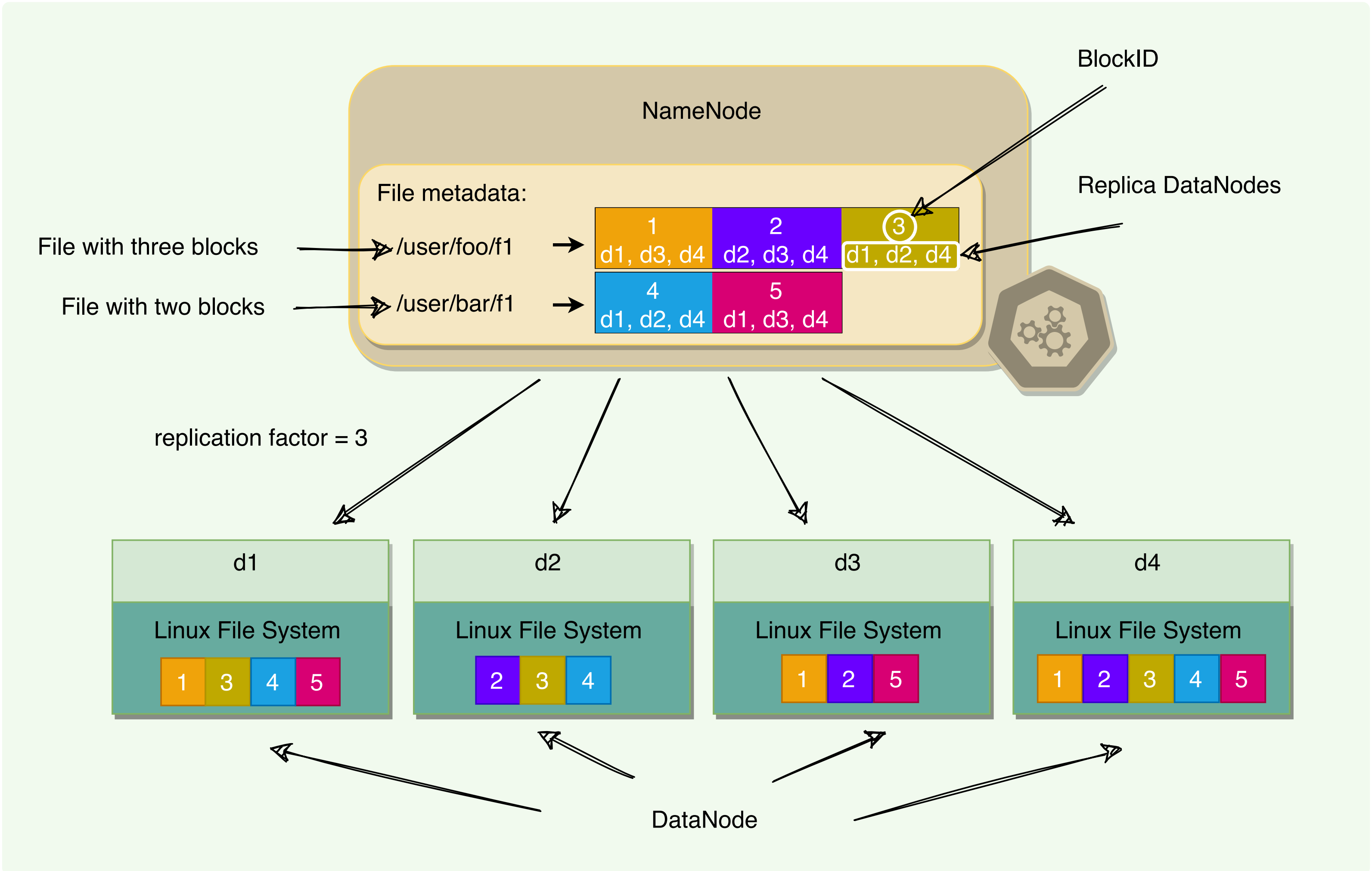
HDFS architecture

All files stored in HDFS are broken into multiple fixed-size blocks, where each block is 128 megabytes in size by default (configurable on a per-file basis). Each file stored in HDFS consists of two parts: the **actual file data** and the **metadata**, i.e., how many block parts the file has, their locations and the total file size, etc. HDFS cluster primarily consists of a **NameNode** that manages the file system metadata and **DataNodes** that store the actual data.


NameNode
In-memory FS Metadata
FsImage
Edit Log
Control flow
metadata request
metadata
Client
Block operations
DataNode Heartbeat
Garbage collection
Block migration
read/write requests
block data
DataNode
Linux File System
Block
Block
...
Block
DataNode
Linux File System
Block
Block
...
Block
Data flow
Data transfer happens directly
between client and DataNodes
Several DataNodes

HDFS high-level architecture

- All blocks of a file are of the same size except the last one.
- HDFS uses **large block sizes** because it is designed to store extremely large files to enable MapReduce jobs to process them efficiently.
- Each block is identified by a unique 64-bit ID called **BlockID**.
- All read/write operations in HDFS operate at the block level.
- DataNodes store each block in a separate file on the local file system and provide read/write access.
- When a DataNode starts up, it scans through its local file system and sends the list of hosted data blocks (called BlockReport) to the NameNode.
- The NameNode maintains two on-disk data structures to store the file system's state: an **FsImage** file and an **EditLog**. FsImage is a checkpoint of the file system metadata at some point in time, while the EditLog is a log of all of the file system metadata transactions since the image file was last created. These two files help NameNode to recover from failure.
- User applications interact with HDFS through its client. HDFS Client interacts with NameNode for metadata, but all data transfers happen directly between the client and DataNodes.
- To achieve high-availability, HDFS creates multiple copies of the data and distributes them on nodes throughout the cluster.


BlockID
NameNode
Replica DataNodes
File metadata:
File with three blocks
/user/foo/f1
1
d1, d3, d4
2
d2, d3, d4
3
d1, d2, d4
File with two blocks
/user/bar/f1
4
d1, d2, d4
5
d1, d3, d4
replication factor = 3
d1
Linux File System
1 3 4 5
d2
Linux File System
2 3 4
d3
Linux File System
1 2 5
d4
Linux File System
1 2 3 4 5
DataNode

HDFS block replication

Comparison between GFS and HDFS

HDFS architecture is similar to GFS, although there are differences in the terminology. Here is the comparison between the two file systems:

	GFS	HDFS
Storage node	ChunkServer	DataNode
File part	Chunk	Block
File part size	Default chunk size is 64MB (adjustable)	Default block size is 128MB (adjustable)
Metadata Checkpoint	Checkpoint image	FsImage
Write ahead log	Operation log	EditLog
Platform	Linux	Cross-Platform
Language	Developed in C++	Developed in Java
Available Implementation	Only used internally by Google	Opensource
Monitoring	Master receives HeartBeat from ChunkServers	NameNode receives HeartBeat from DataNodes
Concurrency	Follows multiple writers and multiple readers model.	Does not support multiple writers. HDFS follows the write-once and read-many model.
File operations	Append and Random writes are possible.	Only append is possible.
Garbage Collection	Any deleted file is renamed into a particular folder to be garbage collected later.	Any deleted file is renamed to a hidden name to be garbage collected later.
Communication	RPC over TCP is used for communication with the master. To minimize latency, pipelining and streaming are used over TCP for data transfer.	RPC over TCP is used for communication with the NameNode. For data transfer, pipelining and streaming are used over TCP.
Cache Management	Clients cache metadata. Client or ChunkServer does not cache file data. ChunkServers rely on the buffer cache in Linux to maintain frequently accessed data in memory.	HDFS uses distributed cache. User-specified paths are cached explicitly in the DataNode's memory in an off-heap block cache. The cache could be private (belonging to one user) or public (belonging to all the users of the same node).
Replication Strategy	Chunk replicas are spread across the racks. Master automatically replicates the chunks. By default, three copies of each chunk are stored. User can specify a different replication factor. The master re-replicates a chunk replica as soon as the number of available replicas falls below a user-specified number.	The HDFS has an automatic rack-ware replication system. By default, two copies of each block are stored at two different DataNodes in the same rack, and a third copy is stored on a Data Node in a different rack (for better reliability). User can specify a different replication factor.
File system Namespace	Files are organized hierarchically in directories and identified by pathnames.	HDFS supports a traditional hierarchical file organization. Users or applications can create directories to store files inside. HDFS also supports third-party file systems such as Amazon Simple Storage Service (S3) and Cloud Store.
Database	Bigtable uses GFS as its storage engine.	HBase uses HDFS as its storage engine.

← Previous
Hadoop Distributed File System: Introduction

Next →
Deep Dive

☐ Mark as Completed